

HMQ: Hardware Friendly Mixed Precision Quantization Block for CNNs

Hai Victor Habi, Roy H. Jennings, and Arnon Netzer

Sony Semiconductor Israel
{hai.habi, roy.jennings, arnon.netzer}@sony.com

July 21, 2020

Abstract

Recent work in network quantization produced state-of-the-art results using mixed precision quantization. An imperative requirement for many efficient edge device hardware implementations is that their quantizers are uniform and with power-of-two thresholds. In this work, we introduce the Hardware Friendly Mixed Precision Quantization Block (HMQ) in order to meet this requirement. The HMQ is a mixed precision quantization block that repurposes the Gumbel-Softmax estimator into a smooth estimator of a pair of quantization parameters, namely, bit-width and threshold. HMQs use this to search over a finite space of quantization schemes. Empirically, we apply HMQs to quantize classification models trained on CIFAR10 and ImageNet. For ImageNet, we quantize four different architectures and show that, in spite of the added restrictions to our quantization scheme, we achieve competitive and, in some cases, state-of-the-art results.

1 Introduction

In recent years, convolutional neural networks (CNNs) produced state-of-the-art results in many computer vision tasks including image classification [14, 17, 21, 22, 38, 39], object detection [29, 36, 40], semantic segmentation [31, 37], etc. Deploying these models on embedded devices is a challenging task due to limitations on available memory, computational power and power consumption.

Many works address these issues using different methods. These include pruning [16, 45, 47], efficient neural architecture design [14, 21, 24, 38], hardware and CNN co-design [14, 20, 43] and quantization [6, 13, 15, 23, 24, 46].

In this work, we focus on quantization, an approach in which the model is compressed by reducing the bit-widths of weights and activations. Besides re-

The code of this work is available in <https://github.com/sony-si/ai-research>.

duction in memory requirements, depending on the specific hardware, quantization usually also results in the reduction of both latency and power consumption. The challenge of quantization is to reduce the model size without compromising its performance. For high compression rates, this is usually achieved by *fine-tuning* a pre-trained model for quantization. In addition, recent work in quantization focused on making quantizers more *hardware friendly* (amenable to deployment on embedded devices) by restricting quantization schemes to be: per-tensor, uniform, symmetric and with thresholds that are powers of two [24, 41].

Recently, *mixed-precision* quantization was studied in [12, 41, 42, 44]. In these works, the bit-widths of weights and activations are not equal across the model and are learned during some optimization process. In [42], reinforcement learning is used, which requires the training of an agent that decides the bit-width of each layer. In [44], neural architecture search is used, which implies duplication of nodes in the network and that the size of the model grows proportionally to the size of the search space of bit-widths. Both of these methods limit the bit-width search space because of their computational cost. In [12], the bit-widths are not searched during training, but rather, this method relies on the relationship between the layer's Hessian and its sensitivity to quantization.

An imperative requirement for many efficient edge device hardware implementations is that their quantizers are symmetric, uniform and with power-of-two thresholds (see [24]). This removes the cost of special handling of zero points and real value scale factors. In this work, we introduce a novel quantization block we call the *Hardware Friendly Mixed Precision Quantization Block* (HMQ) that is designed to search over a finite set of quantization schemes that meet this requirement. HMQs utilize the Gumbel-Softmax estimator [25] in order to optimize over a categorical distribution whose samples correspond to quantization scheme parameters.

We propose a method, based on HMQs, in which both the bit-width and the quantizer's threshold are searched simultaneously. We present state-of-the-art results on MobileNetV1, MobileNetV2 and ResNet-50 in most cases, in spite of the hardware friendly restriction applied to the quantization schemes. Additionally, we present the first (that we know of) mixed precision quantization results of EfficientNet-B0. In particular, our contributions are the following:

- We introduce HMQ, a novel, hardware friendly, mixed precision quantization block which enables a simple and efficient search for quantization parameters.
- We present an optimization method, based on HMQs, for mixed precision quantization in which we search simultaneously for both the bit-width and the threshold of each quantizer.
- We present competitive and, in most cases, state-of-the-art results using our method to quantize ResNet-50, EfficientNet-B0, MobileNetV1 and MobileNetV2 classification models on ImageNet.

2 Related Work

Quantization lies within an active area of research that tries to reduce memory requirements, power consumption and inference latencies of neural networks. These works use techniques such as pruning, efficient network architectures and distillation (see e.g. [7, 14, 15, 18, 19, 21, 30, 34, 35, 38, 39, 48]). Quantization is a key method in this area of research which compresses and accelerates the model by reducing the number of bits used to represent model weights and activations.

Quantization. Quantization techniques can be roughly divided into two families: post-training quantization techniques and quantization-aware training techniques. In post-training quantization techniques, a trained model is quantized without retraining the model (see e.g. [1, 5]). In quantization-aware training techniques, a model undergoes an optimization process during which the model is quantized. A key challenge in this area of research, is to compress the model without significant degradation to its accuracy. Post-training techniques suffer from a higher degradation to accuracy, especially for high compression rates.

Since the gradient of quantization functions is zero almost everywhere, most quantization-aware training techniques use the straight through estimator (STE) [4] for the estimation of the gradients of quantization functions. These techniques mostly differ in their choice of quantizers, the quantizers' parametrization (thresholds, bit-widths, step size, etc.) and their training procedure. During training, the network weights are usually stored in full-precision and are quantized before they are used in feed-forward. The full-precision weights are then updated via back-propagation. Uniform quantizers are an important family of quantizers that have several benefits from a hardware point-of-view (see e.g. [13, 24, 41]). Non-uniform quantizers include clustering, logarithmic quantization and others (see e.g. [3, 33, 46, 49]).

Mixed precision. Recent works on quantization produced state-of-the-art results using mixed precision quantization, that is, quantization in which the bit-widths are not constant across the model (weights and activations). In [42], reinforcement learning is used to determine bit-widths. In [12], second order gradient information is used to determine bit-widths. More precisely, the bit-widths are selected by ordering the network layers using this information. In [41], bit-widths are determined by learnable parameters whose gradients are estimated using STE. This work focuses on the choice of parametrization of the quantizers and shows that the threshold (dynamic range) and step size are preferable over parametrizations that use bit-widths explicitly.

In [44], a mixed precision quantization-aware training technique is proposed where the bit-widths search is converted into a network architecture search (based on [27]). More precisely, in this solution, the search space of all possible quantization schemes is, in fact, a search for a sub-graph in a super-net. The disadvantage of this approach, is that the size of the super net grows substantially with every optional quantization edge/path that is added to the super

net. In practice, this limits the architecture search space. Moreover, this work deals with bit-widths and thresholds as two separate problems where thresholds follow the solution in [8].

3 The HMQ Block

The *Hardware Friendly Mixed Precision Quantization Block* (HMQ) is a network block that learns, via standard SGD, a uniform and symmetric quantization scheme. The scheme is parametrized by a pair (t, b) of threshold t and bit-width b . During training, an HMQ searches for (t, b) over a finite space $T \times B \subseteq \mathbb{R}^+ \times \mathbb{N}$. In this work, we make HMQs “hardware friendly” by also forcing their thresholds to be powers of two. We do this by restricting

$$T = \{2^M, 2^{M-1}, \dots, 2^{M-8}\} \quad (1)$$

where $M \in \mathbb{Z}$ is an integer we configure per HMQ (see Section 4).

The step size Δ of a uniform quantization scheme is the (constant) gap between any two adjacent quantization points. Δ is parametrized by (t, b) differently for a signed quantizer, where $\Delta = \frac{2t}{2^b}$, and an unsigned one, where $\Delta = \frac{t}{2^b}$. Note that Δ ties the bit-width and threshold values into a single parameter but Δ is not uniquely defined by them. The definition of the quantizer that we use in this work is similar to the one in [24]. The signed version Q^s of a quantizer of an HMQ is defined as follows:

$$Q^s(x, \Delta, t) = \text{clip} \left(\Delta \cdot \left\lceil \frac{x}{\Delta} \right\rceil, -(t - \Delta), t \right) \quad (2)$$

where $\text{clip}(x, a, b) = \min(\max(x, a), b)$ and $\lceil x \rceil$ is the rounding function. Similarly, the unsigned version Q^{us} is defined as follows:

$$Q^{\text{us}}(x, \Delta, t) = \text{clip} \left(\Delta \cdot \left\lceil \frac{x}{\Delta} \right\rceil, 0, t - \Delta \right). \quad (3)$$

In the rest of this section we assume that the quantizer Q of an HMQ is signed, but it applies to both signed and unsigned quantizers.

In order to search over a discrete set, the HMQ represents each pair in $T \times B$ as a sample of a categorical random variable of the Gumbel-Softmax estimator (see [25, 32]). This enables the HMQ to search for a pair of threshold and bit-width. The Gumbel-Softmax is a continuous distribution on the simplex that approximates categorical samples. In our case, we use this approximation as a joint discrete probability distribution of thresholds and bit-widths $P_{T,B}(T=t, B=b|g_{t,b})$ on $T \times B$:

$$P_{T,B}(T=t, B=b|g_{t,b}) = \frac{\exp(\frac{\log(\hat{\pi}_{t,b}) + g_{t,b}}{\tau})}{\sum_{t' \in T} \sum_{b' \in B} \exp(\frac{\log(\hat{\pi}_{t',b'}) + g_{t',b'}}{\tau})} \quad (4)$$

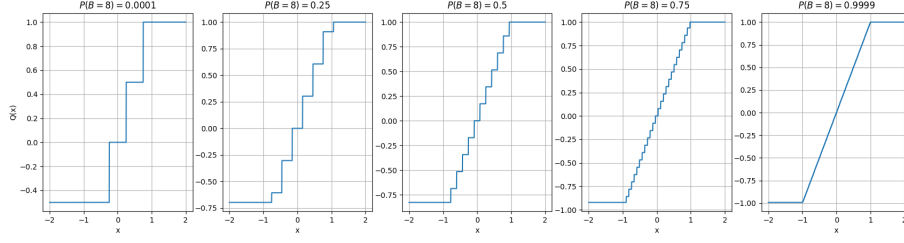


Figure 1: The quantization scheme of an HMQ with $T = \{1\}$ and $B = \{2, 8\}$ for different approximations of the Gumbel-Softmax. Transition from 2-bit quantization $P(B = 8) \approx 0$ (left) to 8-bit quantization $P(B = 8) \approx 1$ (right)

where $\hat{\pi}$ is a matrix of class probabilities whose entries $\hat{\pi}_{t,b}$ correspond to pairs in $T \times B$, $g_{t,b}$ are random i.i.d. variables drawn from $\text{Gumbel}(0, 1)$ and $\tau > 0$ is a softmax temperature value. We define $\hat{\pi} = \text{softmax}(\pi)$ where π is a matrix of trainable parameters $\pi_{t,b}$. This guarantees that the matrix $\hat{\pi}$ forms a categorical distribution.

The quantizers in Equations 2 and 3 are well defined for any two real numbers $\Delta > 0$ and $t > 0$. During training, in feed forward, we sample $g_{t,b}$ and use these samples in the approximation $P_{T,B}$ of a categorical choice. The HMQ parametrizes its quantizer $Q(x, \hat{\Delta}, \hat{t})$ using an **expected step size** $\hat{\Delta}$ and an **expected threshold** $\hat{t}$ that are defined as follows:

$$\hat{\Delta} = \sum_{t \in T} \sum_{b \in B} P_{T,B}(T = t, B = b | g_{t,b}) \cdot \Delta_{t,b}, \quad (5)$$

$$\hat{t} = \sum_{t \in T} P_T(T = t) \cdot t \quad (6)$$

where $P_T(T = t) = \sum_{b' \in B} P_{T,B}(T = t, B = b' | g_{t,b'})$ is the marginal distribution of thresholds and $\Delta_{t,b} = \frac{2t}{2^b}$.

In back-propagation, the gradients of rounding operations are estimated using the STE and the rest of the module, i.e. Equations 4, 5 and 6, are differentiable. This implies that the HMQ smoothly updates the parameters $\pi_{t,b}$ which, in turn, smoothly updates the estimated bit-width and threshold values of the quantization scheme. Figure 1 shows examples of HMQ quantization schemes during training. During inference, the HMQ's quantizer is parametrized by the pair (t, b) that corresponds to the maximal parameter $\pi_{t,b}$.

Note that the temperature parameter τ of the Gumbel-Softmax estimator in Equation 4 has a dual effect during training. As it approaches zero, in addition to approximating a categorical choice of a unique pair $(t, b) \in T \times B$, smaller values of τ also incur a larger variance of gradients which adds instability to the optimization process. This problem is mitigated by annealing τ (see Section 4).

4 Optimization Process

In this section, we present a fine-tuning optimization process that is applied to a full precision, 32-bit floating point, pre-trained model after adding HMQs. Throughout this work, we use the term model weights (or simply weights) to refer to all of the trainable model weights, *not including* the HMQ parameters. We denote by Θ , the set of weight tensors to be quantized; by $\mathcal{X}$, the set of activation tensors to be quantized and by Π , the set of HMQ parameters. Given a tensor T , we use the notation $|T|$ to denote the number of entries in T .

From a high level view, our optimization process consists of two phases. In the first phase, we simultaneously train both the model weights and the HMQ parameters. We take different approaches for quantization of weights and activations. These are described in Sections 4.1 and 4.2. We split the first phase into cycles with an equal number of epochs each. In each cycle of the first phase, we reset the Gumbel-Softmax temperature τ in Equation 4 and anneal it till the end of the cycle. In the second phase of the optimization process, we fine-tune only the model weights. During this phase, similarly to HMQs behaviour during inference, the quantizer of every HMQ is parametrized by the pair (t, b) that corresponds to the maximal parameter $\pi_{t,b}$ that was learnt in the first phase.

4.1 Weight Compression

Let θ be an input tensor of weights to be quantized by some HMQ. We define the set of thresholds T in the search space $T \times B$ of the HMQ by setting M in Equation 1 to be $\min\{M : 2^M \geq \max(\text{abs}(\theta)), i \in \mathbb{Z}\}$. The values in B are different per experiment (see Section 5).

Denote by Π_w the subset of Π containing all of the parameters of HMQs quantizing weights. The expected weight compression rate, induced by the values of Π_w is defined as follows:

$$\tilde{R}(\Pi_w) = \frac{32 \sum_{\theta_i \in \Theta} |\theta_i|}{\sum_{\theta_i \in \Theta} \mathbb{E}[b_i] |\theta_i|} \quad (7)$$

where θ_i is a tensor of weights and $\mathbb{E}[b_i] = \sum_{b \in B} b \cdot P_B^i(B = b)$ is the expected bit-width of θ_i , where P_B^i is the bit-width marginal distribution in the Gumbel-Softmax estimation of the corresponding HMQ. In other words, assuming that all of the model weights are quantized by HMQs, the numerator is the memory requirement of the weights of the model before compression and the denominator is the expected memory requirement during training.

During the first phase of the optimization process, we optimize the model with respect to a target weight compression rate $R_w \in \mathbb{R}^+$, by minimizing (via standard SGD) the following loss function:

$$J(\Theta, \Pi) = J_{task}(\Theta, \Pi) + \lambda (J_w(\Pi_w))^2 \quad (8)$$

where $J_{task}(\Theta, \Pi)$ is the original, task specific loss, e.g. the standard cross entropy loss, $J_w(\Pi_w)$ is a loss with respect to the target compression rate R_w and λ is a hyper-parameter that control the trade-off between the two. We define $J_w(\Pi_w)$ as follows:

$$J_w(\Pi_w) = \frac{\max(0, R_w - \tilde{R}(\Pi_w))}{R_w}. \quad (9)$$

In practice, we gradually increase the target compression rate R_w during the first few cycles in the first phase of our optimization process. This approach of gradual training of quantization is widely used, see e.g. [2, 10, 12, 49]. In most cases, layers are gradually added to the training process whereas in our process we gradually decrease the bit-width across the whole model, albeit, with mixed precision.

By the definition of $J_w(\Pi_w)$, if the target weight compression rate is met during training, i.e. $\tilde{R}(\Pi_w) > R_w$, then the gradients of $J_w(\Pi_w)$ with respect to the parameters in Π_w are zero and the task specific loss function determines the gradients alone. In our experiments, the actual compression obtained by using a specific target compression R_w depends on the hyper-parameter λ and the sensitivity of the architecture to quantization.

4.2 Activations Compression

We define T in the search space $T \times B$ of an HMQ that quantizes a tensor of activations similarly to HMQs quantizing weights. We set $M \in \mathbb{Z}$ in Equation 1 to be minimum such that 2^M is greater or equal than the maximum absolute value of an activation of the pre-trained model over the entire training set.

The objective of activations compression is to fit any single activations tensor, after quantization, into a given size of memory $\bar{U} \in \mathbb{N}$ (number of bits). This objective is inspired by the one in [41] and is especially useful for DNNs in which the operators in the computational graph induce a path graph, i.e. the operators are executed sequentially. We define the target activations compression rate R_a to be

$$R_a = \frac{32 \cdot \max_{X_i \in \mathcal{X}} |X_i|}{\bar{U}} \quad (10)$$

where X_i are the activation tensors to be quantized. Note that $\bar{U}$ implies the precise (maximum) number of bits $b(X)$ of every feature map $X \in \mathcal{X}$:

$$b(X) = \left\lceil \frac{\bar{U}}{|X|} \right\rceil. \quad (11)$$

We assume that $b(X) \geq 1$ for every feature map $X \in \mathcal{X}$ (otherwise, the requirement cannot be met and $\bar{U}$ should be increased) and fix $B = \{\min(b(X), 8)\}$ in the search space of the HMQ that corresponds to X . Note that this method

can also be applied to models with a more complex computational graph, such as ResNet, by applying Equation 11 to blocks instead of single feature maps. Note also, that by definition, the maximum bit-width of *every activation* is 8. We can therefore assume that $R_a \geq 4$.

Here, the bit-widths of every feature map is determined by Equation 11. This is in contrast to the approach in [41] (for activations compression) and our approach for weight compression in Section 4.1, where the choice of bit-widths is a result of an SGD minimization process. This allows a more direct approach for the quantization of activations in which we gradually increase R_a , during the first few cycles in the first phase of the optimization process. In this approach, while activation HMQs learn the thresholds, their bit-widths are implied by R_a . This, in contrast to adding a target activations compression component to the loss, both guarantees that the target compression of activations is obtained and simplifies the loss function of the optimization process.

5 Experimental Results

In this section, we present results using HMQs to quantize various classification models. As proof of concept, we first quantize ResNet-18 [17] trained on CIFAR-10 [26]. For the more challenging ImageNet [9] classification task, we present results quantizing ResNet-50 [17], EfficientNet-B0 [39], MobileNetV1 [21] and MobileNetV2 [38].

In all of our experiments, we perform our fine-tuning process on a full precision, 32-bit floating point, pre-trained model in which an HMQ is added after every weight and every activation tensor per layer, *including the first and last layers*, namely the input convolutional layer and the fully connected layer. The parameters $\pi_{t,b}$ of every HMQ are initialized as a categorical distribution in which the parameter that corresponds to the pair of the maximum threshold with the maximum bit-width is initialized to 0.9 and 0.1 is uniformly distributed between the rest of the parameters. The bit-width set B in the search space of HMQs is set differently for CIFAR-10 and ImageNet (see Sections 5.1 and 5.2).

Note that in all of the experiments, in all of the weight HMQs, the maximal bit-width is 8 (similarly to activation HMQs). This implies that $\hat{R}(\Pi_w) \geq 4$ throughout the fine-tuning process. The optimizer that we use in all of our experiments is RAdam [28] with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. We use different learning rates for the model weights and the HMQ parameters. The data augmentation that we use during fine-tuning is the same as the one used to train the base models.

The entire process is split into two phases, as described in Section 4. The first phase consists of 30 epochs split into 6 cycles of 5 epochs each. In each cycle, the temperature τ in Equation 4, is reset and annealed till the end of the

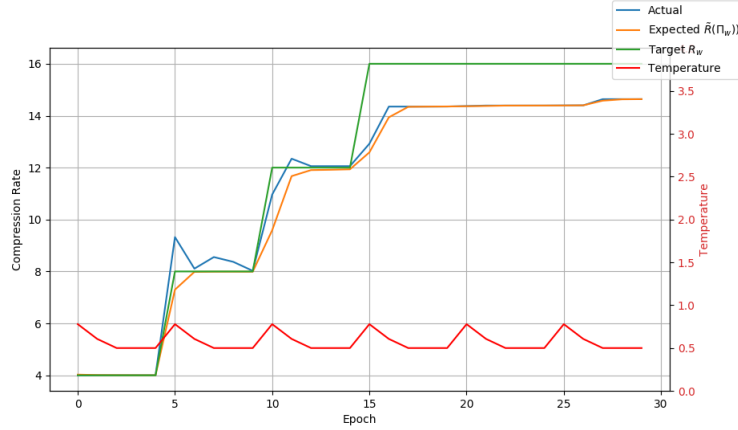


Figure 2: Expected and actual weight compression rates during fine-tuning of MobileNetV2 on ImageNet as the target compression rate and τ are updated

cycle. We update the temperature every N steps within a cycle, where $25 \cdot N$ is the number of steps in a single epoch. The annealing function that we use is similar to the one in [25]:

$$\tau(i) = \max(e^{-ir}, 0.5) \quad (12)$$

where i is the training step (within the cycle) and $r = e^{-2}$. The second phase, in which only weights are fine-tuned, consists of 20 epochs.

As mentioned in Section 4, during the first phase, we gradually increase both the weight and activation target compression rates R_w and R_a , respectively. Both target compression rates are initialized to a minimum compression of 4 (implying 8-bit quantization) and are increased, in equally sized steps, at the beginning of each cycle, during the first 4 cycles.

Figure 2 shows an example of the behaviour of the expected weight compression rate $\tilde{R}(\Pi_w)$ and the actual weight compression rate (implied by the quantization schemes corresponding to the maximum parameters $\pi_{t,b}$) during training, as the value of the target weight compression rate R_w is increased and the temperature τ of the Gumbel-Softmax is annealed in every cycle. Note how the difference between the expected and the actual compression rate values decreases with τ , in every cycle (as to be expected by the Gumbel-Softmax estimator’s behaviour).

We compare our results with those of other quantization methods based on top1 accuracy vs. compression metrics. We use weight compression rate (WCR) to denote the ratio between the total size (number of bits) of the weights in the original model and the total size of the weights in the compressed model. Activation compression rate (ACR) denotes the ratio between the size (number

of bits) of the largest activation tensor in the original model and its size in the compressed model. As explained in Section 4.2, our method guarantees that the size of every single activation tensor in the compressed model is bounded from above by a predetermined value $\bar{U}$.

5.1 ResNet-18 on CIFAR-10

As proof of concept, we use HMQs to quantize a ResNet-18 model that is trained on CIFAR-10 with standard data-augmentation from [17]. Our baseline model has top-1 accuracy of 92.45%. We set $B = \{1, 2, 3, 4, 5, 6, 7, 8\}$ in the search space of HMQs quantizing weights. For activations, B is set according to our method in Section 4.2. In all of the experiments in this section, we set $\lambda = 32$ in the loss function in Equation 8. The learning rate that we use for model weights is $1e-5$. For HMQ parameters the learning rate is $1e3$. The batch-size that we use is 256.

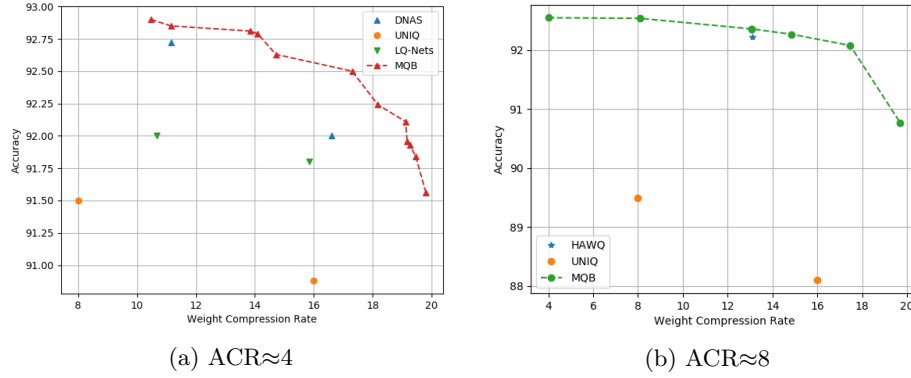


Figure 3: Pareto frontier of weight compression rate vs. top-1 accuracy of ResNet-18 on CIFAR-10 for two Activation Compression Rate (ACR) groups: 4 (Figure 3a) and 8 (Figure 3b) compared with different quantization methods

Figure 3 presents the Pareto frontier of weight compression rate vs. top-1 accuracy for different quantization methods of ResNet-18 on CIFAR-10. In this figure, we show that our method is effective, in comparison to other methods, namely DNAS [44], UNIQ [3], LQ-Nets [46] and HAWQ [12], using different activation compression rates.

We explain our better results, compared to LQ-Nets and UNIQ, in spite of the higher activation and weight compression rates, by the fact that HMQs take advantage of mixed precision quantization. Compared to DNAS, our method has a much larger search space, since in their method, each quantization scheme is translated into a sub-graph in a super net. Moreover, HMQs tie the bit-width and threshold into a single parameter using Equation 5. Comparing our method to HAWQ, HAWQ only uses the Hessian information whereas we perform an optimization over the bit-width.

5.2 ImageNet

In this section, we present results using HMQs to quantize several model architectures, namely MobileNetV1 [21], MobileNetV2 [38], ResNet-50 [17] and EfficientNet-B0 [39] trained on the ImageNet [9] classification dataset. In each of these cases, we use the same data augmentation as the one reported in the corresponding paper. Our baseline models have the following top-1 accuracies: MobileNetV1 (70.6), MobileNetV2 (71.88²), ResNet-50 (76.15²) and EfficientNet-B0 (76.8³). In all of the experiments in this section, we set $B = \{2, 3, 4, 5, 6, 7, 8\}$ in the search space of HMQs quantizing weights. For activations, B is set according to our method in Section 4.2.

As mentioned above, we use the RAdam optimizer in all of our experiments and we use different learning rates for the model weights and the HMQ parameters. For model weights, we use the following learning rates: MobileNetV1 (5e-6), MobileNetV2 (2.5e-6), ResNet-50 (2.5e-6) and EfficientNet-B0 (2.5e-6). For HMQ parameters, the learning rate is equal to the learning rate of the weights multiplied by 1e3. The batch-sizes that we use are: MobileNetV1 (256), MobileNetV2 (128), ResNet-50 (64) and EfficientNet-B0 (128).

5.2.1 Weight Quantization.

In Table 1, we present our results using HMQs to quantize MobileNetV1, MobileNetV2 and ResNet-50. In all of our experiments in this table, we set $R_a = 4$ in Equation 10, implying (single precision) 8-bit quantization of all of the activations. We split the comparison in this table into three compression rate groups: ~ 16 , ~ 10 and ~ 8 in rows 1–2, 3–4 and 5–6, respectively.

Table 1: Weight Compression Rate (WCR) vs. top-1 accuracy (Acc) of MobileNetV1, MobileNetV2 and ResNet-50 on ImageNet. R_w is the target weight compression rate in Equation 9 that was used for fine-tuning

Method	MobileNetV1		MobileNetV2		ResNet-50	
	WCR	Acc	WCR	Acc	WCR	Acc
HAQ [42]	14.8	57.14	14.07	66.75	15.47	70.63
HMQ (ours)	14.15 ($R_w = 16$)	68.36	14.4 ($R_w = 16$)	65.7	15.7 ($R_w = 16$)	75
HAQ	10.22	67.66	9.68	70.9	10.41	75.30
HMQ	10.68 ($R_w = 11$)	69.88	9.71 ($R_w = 10$)	70.12	10.9 ($R_w = 11$)	76.1
HAQ	7.8	71.74	7.46	71.47	8	76.14
HMQ	7.6 ($R_w = 8$)	70.912	7.7 ($R_w = 8$)	71.4	9.01 ($R_w = 9$)	76.3

Note that our method excels in very high compression rates. Moreover, this is in spite of the fact that an HMQ uses uniform quantization and its thresholds are limited to powers of two whereas HAQ uses k-means quantization. We explain our better results by the fact that in HAQ, the bit-widths are the product of a

²Torchvision models (<https://pytorch.org/docs/stable/torchvision/models.html>)

³<https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet>

reinforcement learning agent and the thresholds are determined by the statistics, opposed to HMQs, where they are the product of SGD optimization.

5.2.2 Weight and Activation Quantization.

In Table 2, we compare mixed precision quantization methods in which both weights and activations are quantized. In all of the experiments in this table, the activation compression rate is equal to 8. This means (with some variation between methods) that the smallest number of bits used to quantize activations is equal to 4. This table shows that our method achieves on par results with other mixed precision methods, in spite of the restrictions on the quantization schemes of HMQs. We believe that this is due to the fact that, during training, there is no gradient mismatch for HMQ parameters (see Equations 5 and 6). In other words, HMQs allow smooth propagation of gradients. Additionally, HMQs tie each pair of bit-width and threshold in their search space with a single trainable parameter (opposed to determining the two separately).

Table 2: Comparing Activation Compression Rate (ACR), Weight Compression Rate (WCR) and top-1 accuracy (Acc) of MobileNetV2 and ResNet-50 on ImageNet using different mixed precision quantization techniques. Under ACR: for HAWQ and HAWQ-V2, 8 means that the maximum compression obtained for a single activation tensor is 8. For DQ and HMQ, 8 means that the compression of the largest activation tensor is 8

(a) MobileNetV2				(b) ResNet-50			
Method	ACR	WCR	Acc	Method	ACR	WCR	Acc
DQ [41]	8.05	8.53	69.74	HAWQ [42]	8	12.28	75.3
HMQ _(R_w=8) (ours)	8	8.05	70.9	HAWQ-V2 [11]	8	12.24	75.7
				HMQ _(R_w=13) (ours)	8	13.1	75.45

5.2.3 EfficientNet.

In Table 3, we present results quantizing EfficientNet-B0 using HMQs and in Figure 4, we use the Pareto frontier of accuracy vs model size to summarize our results on all four of the models that were mentioned in this section.

Table 3: Weight Compression Rate (WCR) vs. top-1 accuracy (Acc) of EfficientNetB0 on ImageNet using HMQ quantization. An Activation Compression Rate (ACR) of 4 means single precision 8-bit quantization of activation tensors. R_w is the target weight compression rate that was used during fine-tuning

ACR	R_w	WCR	Acc
4	4	4	76.4
	8	8.05	76
	12	11.97	74.6
	16	14.87	71.54

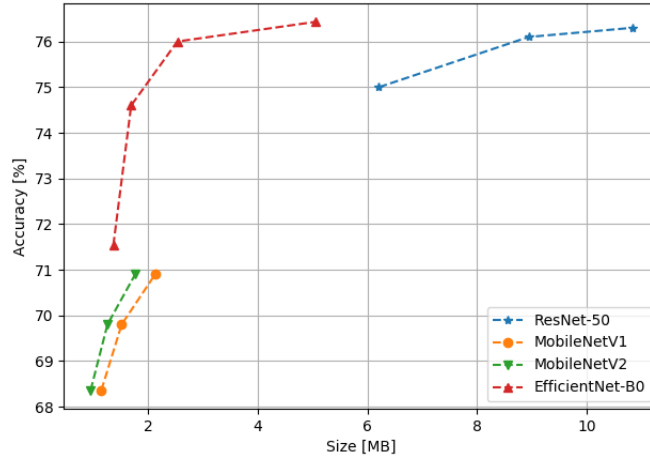


Figure 4: Pareto frontier of top-1 accuracy vs. model size of MobileNetV1, MobileNetV2, ResNet-50 and EfficientNet-B0 quantization by HMQ

5.2.4 Additional Results.

In Figure 5, we present an example of the final bit-widths of weights and activations in MobileNetV1 quantized by HMQ. This figure implies that point-wise convolutions are less sensitive to quantization, compared to their corresponding depth-wise convolutions. Moreover, it seems that deeper layers are also less sensitive to quantization. Note that the bit-widths of activations in Figure 5b are not a result of fine-tuning but are pre-determined by the target activation compression, as described in Section 4.2. In Table 4, we present additional results using HMQs to quantize models trained on ImageNet. This table extends the results in Table 1, here, both weights and activations are quantized using HMQs.

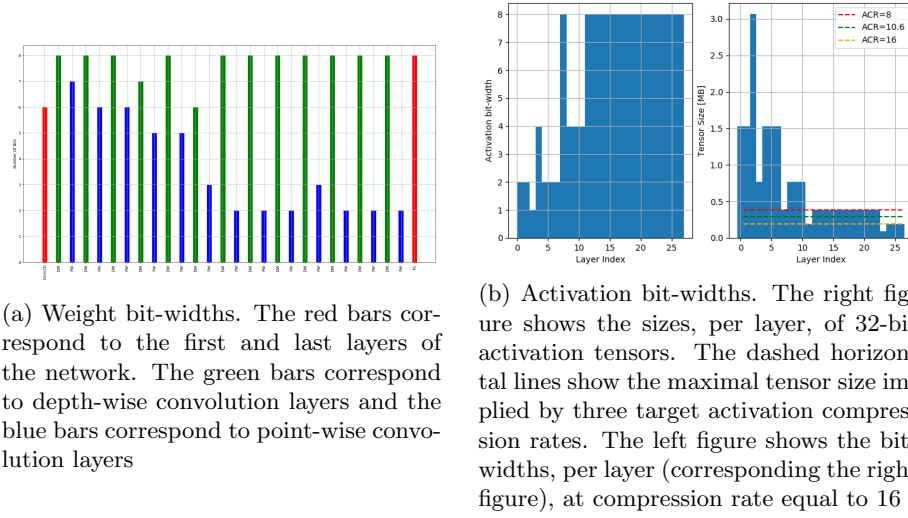


Figure 5: Example of the final bit-width of weights and activations in MobileNetV1 quantized by HMQ

Table 4: Weight Compression Rate (WCR) vs. top-1 accuracy (Acc) of MobileNet-V1, MobileNet-V2 and ResNet50 on ImageNet using HMQ quantization with various target weight compression rates R_w and a fixed Activation Compression Rate (ACR) of 8. MP means Mixed Precision

(a) MobileNetV1				(b) MobileNetV2			
R_w	ACR	WCR	Acc	R_w	ACR	WCR	Acc
16	8 _{MP}	14.638 _{MP}	67.9	16	8 _{MP}	14.8 _{MP}	64.47
11	8 _{MP}	10.709 _{MP}	69.3	10	8 _{MP}	10 _{MP}	69.9

(c) ResNet50			
R_w	ACR	WCR	Acc
16	8 _{MP}	15.45 _{MP}	74.5
11	8 _{MP}	11.1 _{MP}	75.73

6 Conclusions

In this work, we introduced the HMQ, a novel quantization block that can be applied to weights and activations. The HMQ repurposes the Gumbel-Softmax estimator in order to smoothly search over a finite set of uniform and symmetric activation schemes. We presented a standard SGD fine-tuning process, based on

HMQs, for mixed precision quantization that achieves state-of-the-art results in accuracy vs. compression for various networks. Both the model weights and the quantization parameters are trained during this process. This method can facilitate different hardware requirements, including memory, power and inference speed by configuring the HMQ’s search space and the loss function. Empirically, we experimented with two image classification datasets: CIFAR-10 and ImageNet. For ImageNet, we presented state-of-the-art results on MobileNetV1, MobileNetV2 and ResNet-50 in most cases. Additionally, we presented the first (that we know of) quantization results of EfficientNet-B0.

Acknowledgments

We would like to thank Idit Diamant and Oranit Dror for many helpful discussions and suggestions.

References

- [1] R. Banner, Y. Nahshan, and D. Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Advances in Neural Information Processing Systems*, pages 7948–7956, 2019.
- [2] C. Baskin, N. Liss, Y. Chai, E. Zheltonozhskii, E. Schwartz, R. Giryes, A. Mendelson, and A. M. Bronstein. Nice: Noise injection and clamping estimation for neural network quantization. *arXiv preprint arXiv:1810.00162*, 2018.
- [3] C. Baskin, E. Schwartz, E. Zheltonozhskii, N. Liss, R. Giryes, A. M. Bronstein, and A. Mendelson. Uniq: Uniform noise injection for non-uniform quantization of neural networks. *arXiv preprint arXiv:1804.10969*, 2018.
- [4] Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [5] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer. Zeroq: A novel zero shot quantization framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13169–13178, 2020.
- [6] Z. Cai, X. He, J. Sun, and N. Vasconcelos. Deep learning with low precision by half-wave gaussian quantization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5918–5926, 2017.
- [7] G. Chen, W. Choi, X. Yu, T. Han, and M. Chandraker. Learning efficient object detection models with knowledge distillation. In *Advances in Neural Information Processing Systems*, pages 742–751, 2017.

- [8] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [10] Y. Dong, R. Ni, J. Li, Y. Chen, J. Zhu, and H. Su. Learning accurate low-bit deep neural networks with stochastic quantization. *arXiv preprint arXiv:1708.01001*, 2017.
- [11] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. *arXiv preprint arXiv:1911.03852*, 2019.
- [12] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 293–302, 2019.
- [13] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha. Learned step size quantization. *arXiv preprint arXiv:1902.08153*, 2019.
- [14] A. Gholami, K. Kwon, B. Wu, Z. Tai, X. Yue, P. Jin, S. Zhao, and K. Keutzer. Squeezenext: Hardware-aware neural network design. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 1638–1647, 2018.
- [15] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- [16] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. In *Advances in neural information processing systems*, pages 1135–1143, 2015.
- [17] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [18] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4340–4349, 2019.
- [19] Y. He, X. Zhang, and J. Sun. Channel pruning for accelerating very deep neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1389–1397, 2017.

- [20] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1314–1324, 2019.
- [21] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [22] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018.
- [23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- [24] S. R. Jain, A. Gural, M. Wu, and C. Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019.
- [25] E. Jang, S. Gu, and B. Poole. Categorical reparametrization with gumble-softmax. In *International Conference on Learning Representations (ICLR 2017)*. OpenReview. net, 2017.
- [26] A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009.
- [27] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations*, 2019.
- [28] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han. On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*, 2020.
- [29] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016.
- [30] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2736–2744, 2017.
- [31] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.

- [32] C. J. Maddison, A. Mnih, and Y. W. Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations*, 2017.
- [33] D. Miyashita, E. H. Lee, and B. Murmann. Convolutional neural networks using logarithmic data representation. *arXiv preprint arXiv:1603.01025*, 2016.
- [34] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz. Pruning convolutional neural networks for resource efficient inference. In *International Conference on Learning Representations*, 2017.
- [35] A. Polino, R. Pascanu, and D. Alistarh. Model compression via distillation and quantization. In *International Conference on Learning Representations*, 2018.
- [36] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*, pages 91–99, 2015.
- [37] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- [38] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.
- [39] M. Tan and Q. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pages 6105–6114, 2019.
- [40] M. Tan, R. Pang, and Q. V. Le. Efficientdet: Scalable and efficient object detection. *arXiv preprint arXiv:1911.09070*, 2019.
- [41] S. Uhlich, L. Mauch, F. Cardinaux, K. Yoshiyama, J. A. Garcia, S. Tiedemann, T. Kemp, and A. Nakamura. Mixed precision dnns: All you need is a good parametrization. In *International Conference on Learning Representations*, 2020.
- [42] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han. Haq: Hardware-aware automated quantization with mixed precision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8612–8620, 2019.
- [43] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *Proceedings of the IEEE*

- Conference on Computer Vision and Pattern Recognition*, pages 10734–10742, 2019.
- [44] B. Wu, Y. Wang, P. Zhang, Y. Tian, P. Vajda, and K. Keutzer. Mixed precision quantization of convnets via differentiable neural architecture search. *arXiv preprint arXiv:1812.00090*, 2018.
 - [45] R. Yu, A. Li, C.-F. Chen, J.-H. Lai, V. I. Morariu, X. Han, M. Gao, C.-Y. Lin, and L. S. Davis. Nisp: Pruning networks using neuron importance score propagation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 9194–9203, 2018.
 - [46] D. Zhang, J. Yang, D. Ye, and G. Hua. Lq-nets: Learned quantization for highly accurate and compact deep neural networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 365–382, 2018.
 - [47] T. Zhang, S. Ye, K. Zhang, J. Tang, W. Wen, M. Fardad, and Y. Wang. A systematic dnn weight pruning framework using alternating direction method of multipliers. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 184–199, 2018.
 - [48] X. Zhang, X. Zhou, M. Lin, and J. Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6848–6856, 2018.
 - [49] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen. Incremental network quantization: Towards lossless cnns with low-precision weights. In *International Conference on Learning Representations*, 2017.